Pizza Analysis

March 27, 2026

1 Pizza Sales Analysis

[24]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import plotly.express as px
```

2 Import the Required Libraries

[25]:
```
Pizza_data=pd.read_csv("pizza_sales.csv")
Pizza_data.head()
```

[25]:
```
   pizza_id  order_id  pizza_name_id  quantity  order_date order_time  \
0         1         1     hawaiian_m         1  01-01-2015   11:38:36
1         2         2  classic_dlx_m         1  01-01-2015   11:57:40
2         3         2  five_cheese_l         1  01-01-2015   11:57:40
3         4         2    ital_supr_l         1  01-01-2015   11:57:40
4         5         2     mexicana_m         1  01-01-2015   11:57:40

   unit_price  total_price pizza_size pizza_category  \
0       13.25        13.25          M        Classic
1       16.00        16.00          M        Classic
2       18.50        18.50          L         Veggie
3       20.75        20.75          L        Supreme
4       16.00        16.00          M         Veggie

                                 pizza_ingredients  \
0          Sliced Ham, Pineapple, Mozzarella Cheese
1  Pepperoni, Mushrooms, Red Onions, Red Peppers,…
2  Mozzarella Cheese, Provolone Cheese, Smoked Go…
3  Calabrese Salami, Capocollo, Tomatoes, Red Oni…
4  Tomatoes, Red Peppers, Jalapeno Peppers, Red O…

                 pizza_name
0        The Hawaiian Pizza
```

```
1   The Classic Deluxe Pizza
2      The Five Cheese Pizza
3  The Italian Supreme Pizza
4         The Mexicana Pizza
```

[26]:
```
Pizza_data.tail()
```

[26]:
```
       pizza_id  order_id  pizza_name_id  quantity  order_date order_time  \
48615     48616     21348  ckn_alfredo_m         1  31-12-2015   21:23:10
48616     48617     21348  four_cheese_l         1  31-12-2015   21:23:10
48617     48618     21348   napolitana_s         1  31-12-2015   21:23:10
48618     48619     21349     mexicana_l         1  31-12-2015   22:09:54
48619     48620     21350      bbq_ckn_s         1  31-12-2015   23:02:05

       unit_price  total_price pizza_size pizza_category  \
48615       16.75        16.75          M        Chicken
48616       17.95        17.95          L         Veggie
48617       12.00        12.00          S        Classic
48618       20.25        20.25          L         Veggie
48619       12.75        12.75          S        Chicken

                                     pizza_ingredients  \
48615  Chicken, Red Onions, Red Peppers, Mushrooms, A…
48616  Ricotta Cheese, Gorgonzola Piccante Cheese, Mo…
48617  Tomatoes, Anchovies, Green Olives, Red Onions,…
48618  Tomatoes, Red Peppers, Jalapeno Peppers, Red O…
48619  Barbecued Chicken, Red Peppers, Green Peppers,…

                      pizza_name
48615   The Chicken Alfredo Pizza
48616       The Four Cheese Pizza
48617        The Napolitana Pizza
48618          The Mexicana Pizza
48619  The Barbecue Chicken Pizza
```

[27]:
```
print("The Metadata of the dataset: ", Pizza_data.shape)
```

```
The Metadata of the dataset:  (48620, 12)
```

[28]:
```
print("The Rows of the dataset: ", Pizza_data.shape[0])
```

```
The Rows of the dataset:  48620
```

[29]:
```
print("The Columns of the dataset: ", Pizza_data.shape[1])
```

```
The Columns of the dataset:  12
```

[30]:
```
Pizza_data.columns
```

[30]:
```
Index(['pizza_id', 'order_id', 'pizza_name_id', 'quantity', 'order_date',
       'order_time', 'unit_price', 'total_price', 'pizza_size',
       'pizza_category', 'pizza_ingredients', 'pizza_name'],
      dtype='object')
```

[31]:
```
Pizza_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48620 entries, 0 to 48619
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   pizza_id           48620 non-null  int64
 1   order_id           48620 non-null  int64
 2   pizza_name_id      48620 non-null  object
 3   quantity           48620 non-null  int64
 4   order_date         48620 non-null  object
 5   order_time         48620 non-null  object
 6   unit_price         48620 non-null  float64
 7   total_price        48620 non-null  float64
 8   pizza_size         48620 non-null  object
 9   pizza_category     48620 non-null  object
 10  pizza_ingredients  48620 non-null  object
 11  pizza_name         48620 non-null  object
dtypes: float64(2), int64(3), object(7)
memory usage: 4.5+ MB
```

2.0.1 Data Types in Raw Data

[32]:
```
Pizza_data.dtypes
```

[32]:
```
pizza_id               int64
order_id               int64
pizza_name_id         object
quantity               int64
order_date            object
order_time            object
unit_price           float64
total_price          float64
pizza_size            object
pizza_category        object
pizza_ingredients     object
pizza_name            object
dtype: object
```

[33]:
```
Pizza_data.describe()
```

[33]:

	pizza_id	order_id	quantity	unit_price	total_price
count	48620.000000	48620.000000	48620.000000	48620.000000	48620.000000
mean	24310.500000	10701.479761	1.019622	16.494132	16.821474
std	14035.529381	6180.119770	0.143077	3.621789	4.437398
min	1.000000	1.000000	1.000000	9.750000	9.750000
25%	12155.750000	5337.000000	1.000000	12.750000	12.750000
50%	24310.500000	10682.500000	1.000000	16.500000	16.500000
75%	36465.250000	16100.000000	1.000000	20.250000	20.500000
max	48620.000000	21350.000000	4.000000	35.950000	83.000000

2.0.2 KPI'S

[34]:
```
total_revenue = Pizza_data['total_price'].sum()
total_pizzas_sold = Pizza_data['quantity'].sum()
total_orders = Pizza_data['order_id'].nunique()
avg_order_value = total_revenue / total_orders
avg_pizzas_per_order = total_pizzas_sold/ total_orders

print(f"Total Revenue: ${total_revenue:,.2f}")
print(f"Total Pizzas Sold: {total_pizzas_sold:,}")
print(f"Total Orders: {total_orders:,}")
print(f"Avg Order Value: ${avg_order_value:,.2f}")
print(f"Average Pizza per Order : {avg_pizzas_per_order:,.2f}")
```

```
Total Revenue: $817,860.05
Total Pizzas Sold: 49,574
Total Orders: 21,350
Avg Order Value: $38.31
Average Pizza per Order : 2.32
```

2.0.3 Chart's

Ingredient Analysis

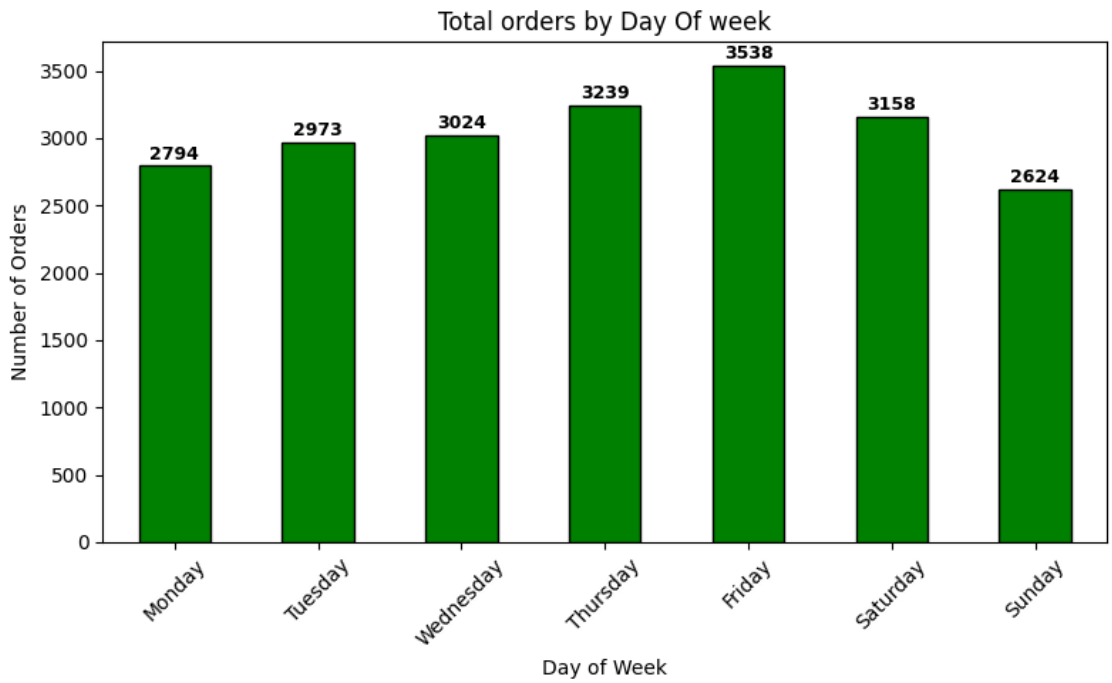
[35]:
```
ingredient = (
                Pizza_data['pizza_ingredients']
                .str.split(',')
                .explode()
                .str.strip()
                .value_counts()
                .reset_index()
                .rename(columns={'index':'Counts','pizza_ingredients':
 ↪'Ingredients'})
)
print(ingredient.head(10))
```

```
         Ingredients  count
0             Garlic  27422
1           Tomatoes  26601
2         Red Onions  19547
```

```
3        Red Peppers  16284
4  Mozzarella Cheese  10333
5          Pepperoni  10300
6            Spinach  10012
7          Mushrooms   9624
8            Chicken   8443
9          Capocollo   6572
```

Daily Trend- Total Orders

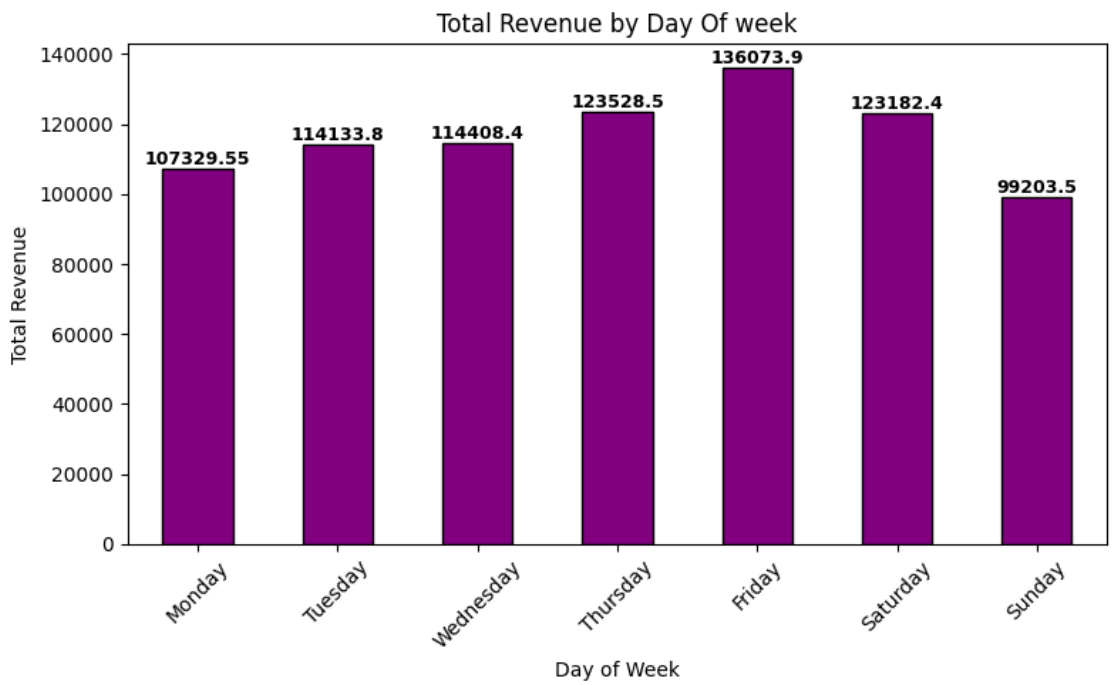
[36]:
```
Pizza_data['order_date'] = pd.to_datetime(Pizza_data['order_date'],␣
↪dayfirst=True)

Pizza_data['day_name'] = Pizza_data['order_date'].dt.day_name()

weekday_order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday",␣
↪"Saturday", "Sunday"]

Pizza_data['day_name'] = pd.Categorical(Pizza_data['day_name'],␣
↪categories=weekday_order, ordered=True)

orders_by_day = Pizza_data.groupby('day_name', observed=False)['order_id'].
↪nunique()

ax = orders_by_day.plot(kind='bar', figsize=(8,5), color='green',␣
↪edgecolor='black')

plt.title("Total orders by Day Of week")
plt.xlabel("Day of Week")
plt.ylabel("Number of Orders")
plt.xticks(rotation=45)

for i, val in enumerate(orders_by_day):
    plt.text(i, val + 20, str(val), ha='center', va='bottom', fontsize=9,␣
↪fontweight='bold')

plt.tight_layout()
plt.show()
```



Daily Trend- Total Revenue

[37]:
```
Pizza_data['order_date'] = pd.to_datetime(Pizza_data['order_date'],␣
↪dayfirst=True)

Pizza_data['day_name'] = Pizza_data['order_date'].dt.day_name()

weekday_order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday",␣
↪"Saturday", "Sunday"]

Pizza_data['day_name'] = pd.Categorical(Pizza_data['day_name'],␣
↪categories=weekday_order, ordered=True)

orders_by_day = Pizza_data.groupby('day_name', observed=False)['total_price'].
↪sum()

ax = orders_by_day.plot(kind='bar', figsize=(8,5), color='purple',␣
↪edgecolor='black')

plt.title("Total Revenue by Day Of week")
plt.xlabel("Day of Week")
plt.ylabel("Total Revenue")
plt.xticks(rotation=45)

for i, val in enumerate(orders_by_day):
```

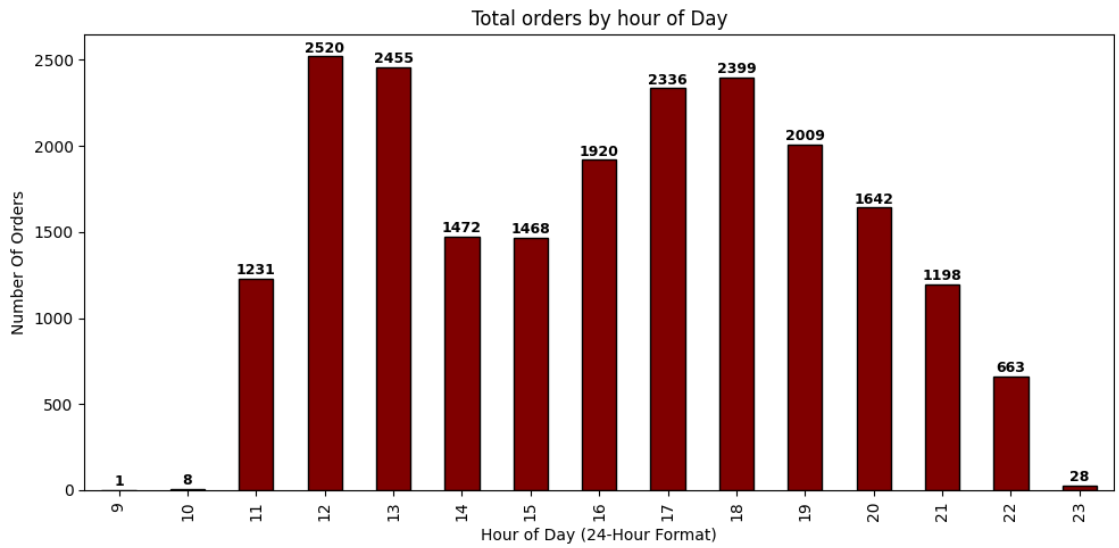
```
    plt.text(i, val + 20, str(val), ha='center', va='bottom', fontsize=9,␣
↪fontweight='bold')

plt.tight_layout()
plt.show()
```



Hourly Trend- Total Orders

[38]:
```
Pizza_data['order_time'] = pd.to_datetime(Pizza_data['order_time'], format='%H:
↪%M:%S')

Pizza_data['order_hour'] = Pizza_data['order_time'].dt.hour

orders_by_hour = Pizza_data.groupby('order_hour', observed=False)['order_id'].
↪nunique()

ax = orders_by_hour.plot(kind='bar', figsize=(10,5), color='maroon',␣
↪edgecolor='black')

plt.title("Total orders by hour of Day")
plt.xlabel("Hour of Day (24-Hour Format)")
plt.ylabel("Number Of Orders")
plt.xticks()

for i, val in enumerate(orders_by_hour):
```

```
    plt.text(i, val + 5, str(val), ha='center', va='bottom', fontsize=9,␣
 ↪fontweight='bold')

plt.tight_layout()
plt.show()
```

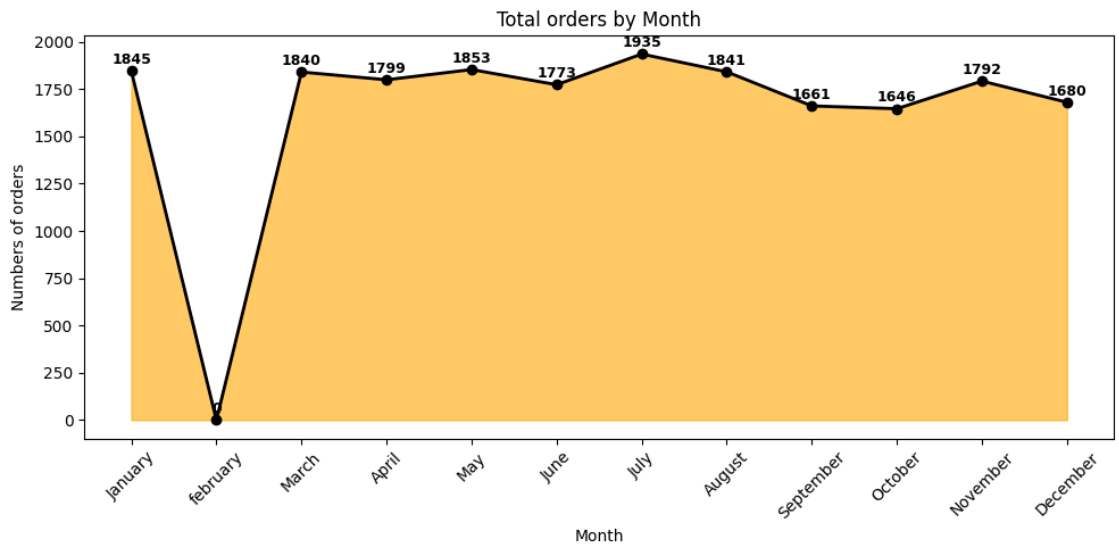


Monthly Trend- Total Orders

[ ]:

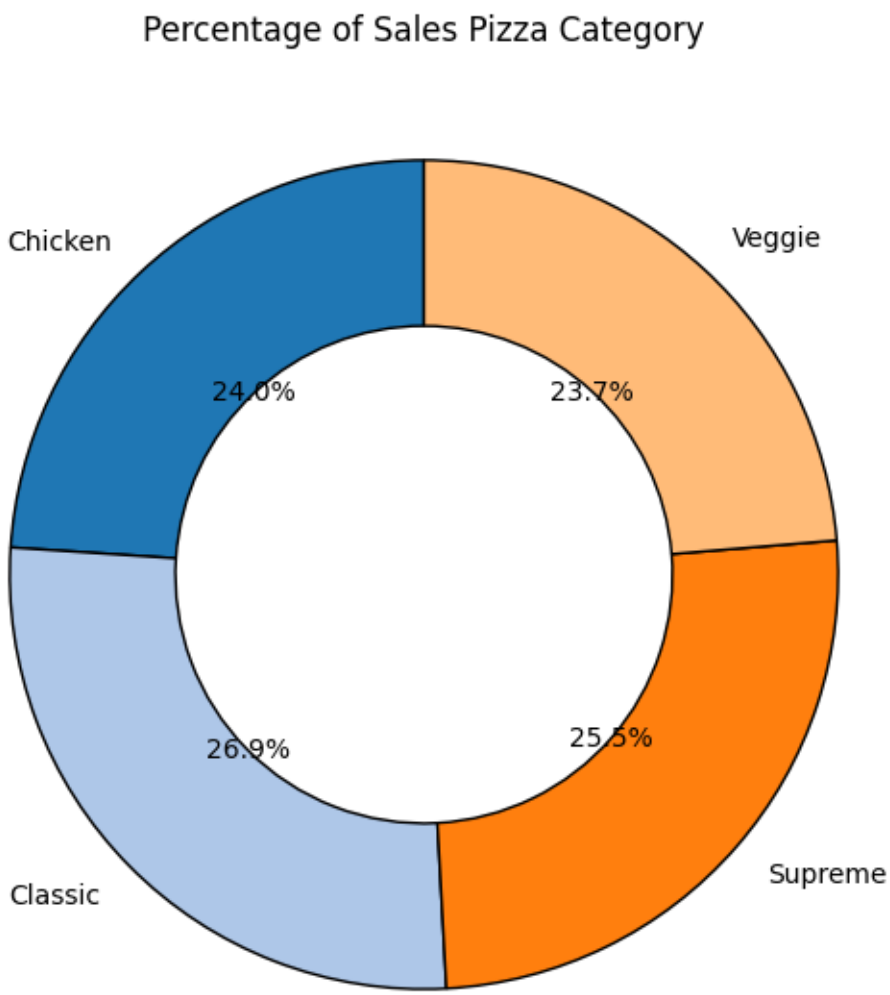
```
[39]: Pizza_data['order_date'] = pd.to_datetime(Pizza_data['order_date'],␣
 ↪dayfirst=True)

Pizza_data['month_name'] = Pizza_data['order_date'].dt.month_name()

month_order = ["January", "february", "March", "April", "May", "June", "July",␣
 ↪"August", "September",
              "October", "November", "December"]

Pizza_data['month_name'] = pd.Categorical(Pizza_data['month_name'],␣
 ↪categories=month_order, ordered=True)

orders_by_month = Pizza_data.groupby('month_name', observed=False)['order_id'].
 ↪nunique()

plt.figure(figsize=(10,5))
plt.fill_between(orders_by_month.index, orders_by_month.values, color="orange",␣
 ↪alpha=0.6)
```

```
plt.plot(orders_by_month.index, orders_by_month.values, color="black",␣
↪linewidth=2, marker='o')

plt.title("Total orders by Month")
plt.xlabel("Month")
plt.ylabel("Numbers of orders")
plt.xticks(rotation=45)

for i, val in enumerate(orders_by_month):
    plt.text(i, val + 20, str(val), ha='center', va='bottom', fontsize=9,␣
↪fontweight='bold')

plt.tight_layout()
plt.show()
```



% of Sales by Category

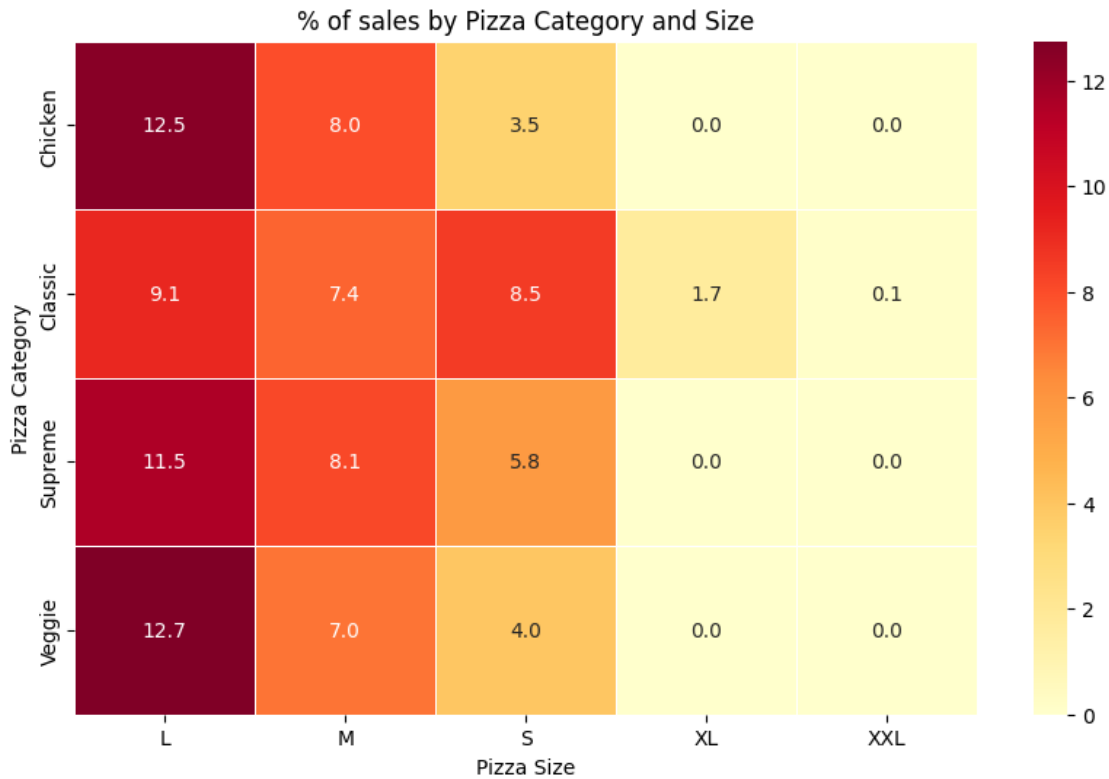
[40]:
```
category_sales = Pizza_data.groupby('pizza_category')['total_price'].sum()

category_pct = category_sales / category_sales.sum() * 100

plt.figure(figsize=(7,7))
colors = plt.get_cmap('tab20').colors #nice color palette

plt.pie(category_pct, labels=category_pct.index, autopct='%1.1f%%',␣
↪startangle=90, colors=colors, wedgeprops={'edgecolor':'black', 'width':0.4})
plt.title("Percentage of Sales Pizza Category")
plt.show()
```



% of Sales by Pizza Size And Category

[41]:
```
sales_pivot = Pizza_data.pivot_table(
    index='pizza_category',
    columns='pizza_size',
    values='total_price',
    aggfunc='sum',
    fill_value=0
)

sales_pct = sales_pivot / sales_pivot.sum().sum() * 100

plt.figure(figsize=(10,6))
```

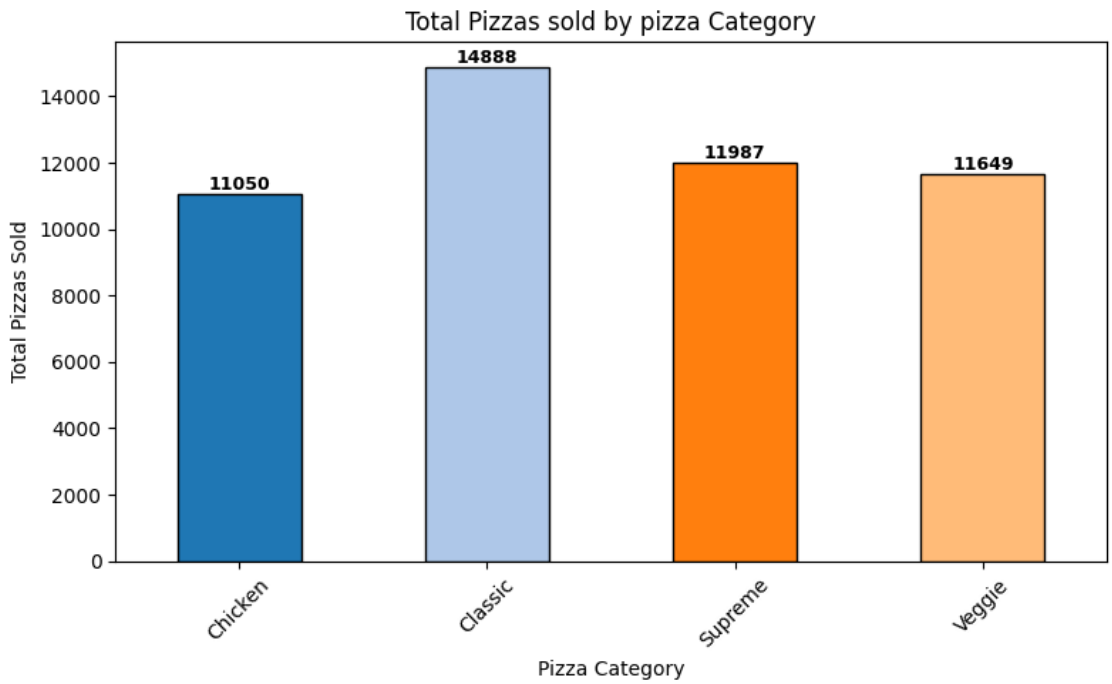
```
sns.heatmap(sales_pct, annot=True, fmt=".1f", cmap="YlOrRd", linewidths=0.5)
plt.title("% of sales by Pizza Category and Size")
plt.ylabel("Pizza Category")
plt.xlabel("Pizza Size")
plt.show()
```



Total Pizzas sold by Pizza Category

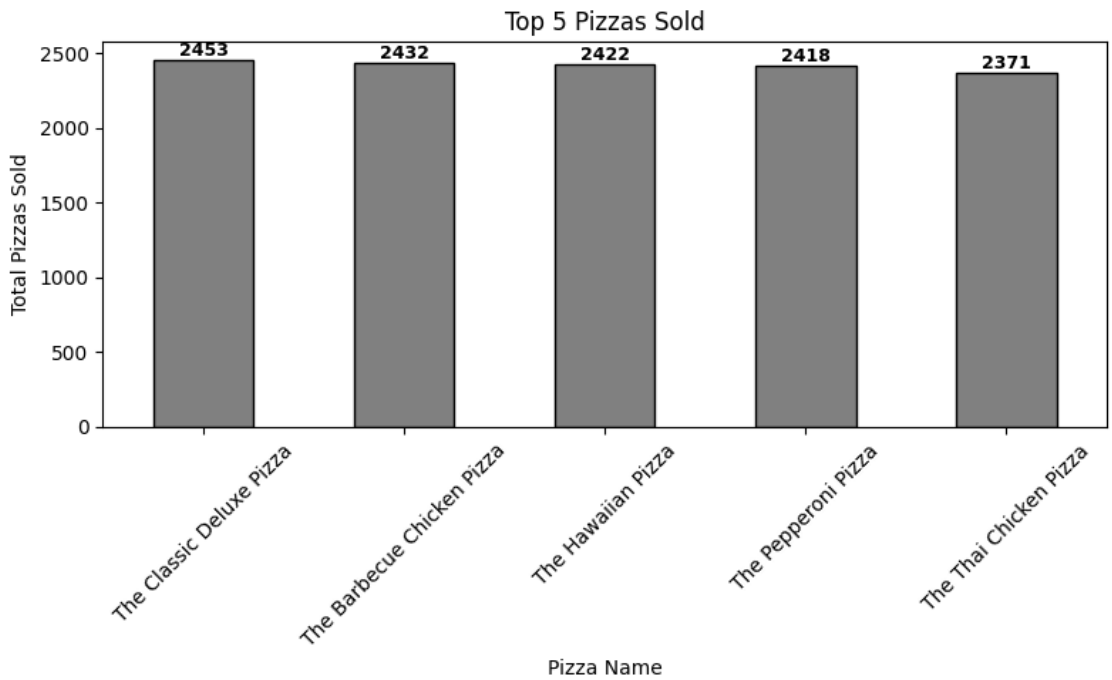
[42]:
```
pizzas_by_category = Pizza_data.groupby('pizza_category')['quantity'].sum()

colors = list(plt.get_cmap('tab20').colors)

colors = colors[:len(pizzas_by_category)]

ax = pizzas_by_category.plot(kind='bar', figsize=(8,5), color=colors,␣
 ↪edgecolor='black')

plt.title("Total Pizzas sold by pizza Category")
plt.xlabel("Pizza Category")
plt.ylabel("Total Pizzas Sold")
plt.xticks(rotation=45)
```

```
for i, val in enumerate(pizzas_by_category):
    plt.text(i, val + 5, str(val), ha='center', va='bottom', fontsize=9,␣
 ↪fontweight='bold')

plt.tight_layout()
plt.show()
```



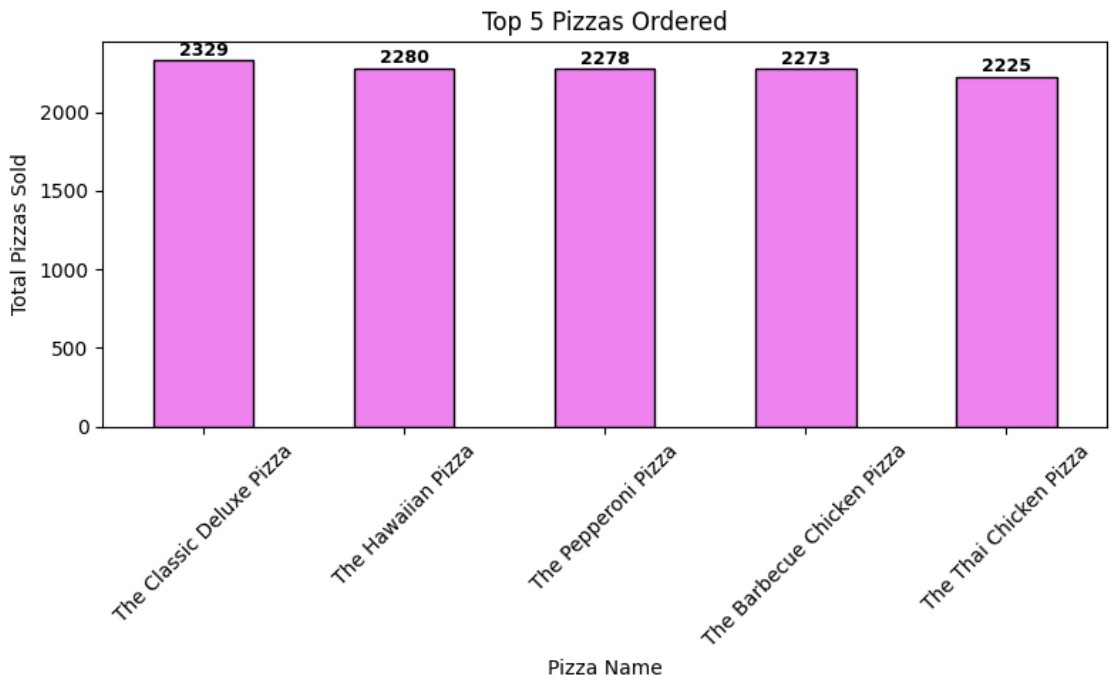
Top 5 Best Selling Pizzas - Total Oty

[43]:
```
pizzas_by_name = Pizza_data.groupby('pizza_name')['quantity'].sum()

top5 = pizzas_by_name.sort_values(ascending=False).head(5)

ax = top5.plot(kind='bar', figsize=(8,5), color='grey', edgecolor='black')
plt.title("Top 5 Pizzas Sold")
plt.xlabel("Pizza Name")
plt.ylabel("Total Pizzas Sold")
plt.xticks(rotation=45)

for i, val in enumerate(top5):
    plt.text(i, val + 2, str(val), ha='center', va='bottom', fontsize=9,␣
 ↪fontweight='bold')

plt.tight_layout()
plt.show()
```



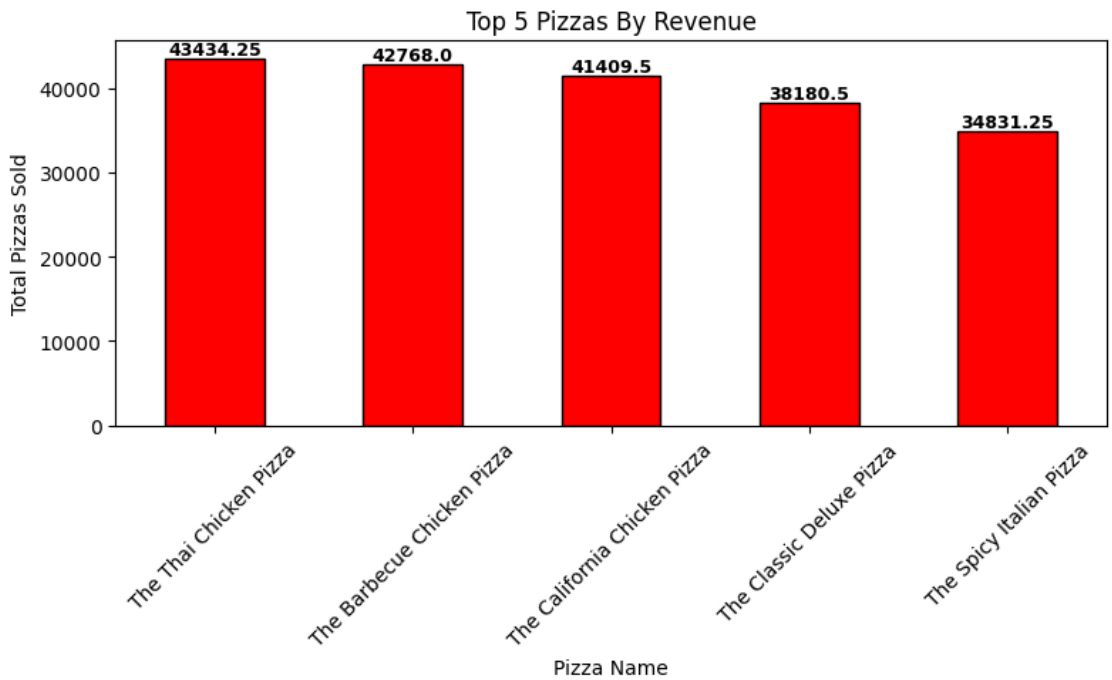
Top 5 Best Selling Pizzas - Total Orders

[44]:
```
pizzas_by_name = Pizza_data.groupby('pizza_name')['order_id'].nunique()

top5 = pizzas_by_name.sort_values(ascending=False).head(5)

ax = top5.plot(kind='bar', figsize=(8,5), color='violet', edgecolor='black')
plt.title("Top 5 Pizzas Ordered")
plt.xlabel("Pizza Name")
plt.ylabel("Total Pizzas Sold")
plt.xticks(rotation=45)

for i, val in enumerate(top5):
    plt.text(i, val + 2, str(val), ha='center', va='bottom', fontsize=9,␣
 ↪fontweight='bold')

plt.tight_layout()
plt.show()
```



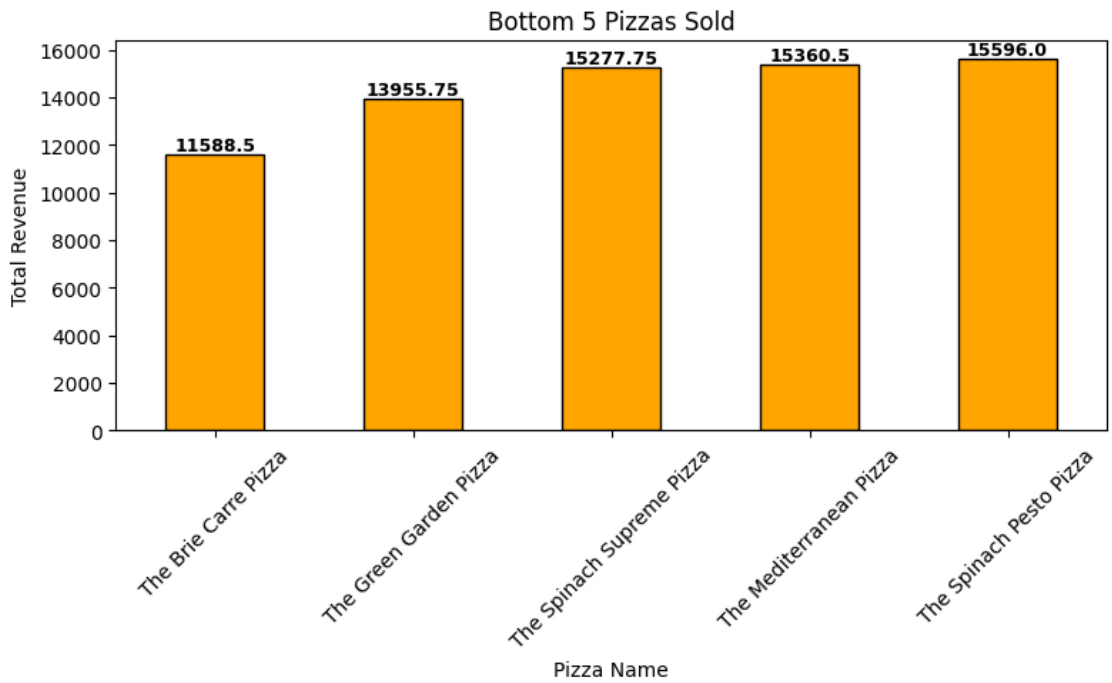
Top 5 Best Selling Pizzas - Total Sales

[45]:
```
pizzas_by_name = Pizza_data.groupby('pizza_name')['total_price'].sum()

top5 = pizzas_by_name.sort_values(ascending=False).head(5)

ax = top5.plot(kind='bar', figsize=(8,5), color='red', edgecolor='black')
plt.title("Top 5 Pizzas By Revenue")
plt.xlabel("Pizza Name")
plt.ylabel("Total Pizzas Sold")
plt.xticks(rotation=45)

for i, val in enumerate(top5):
    plt.text(i, val + 2, str(val), ha='center', va='bottom', fontsize=9,␣
 ↪fontweight='bold')

plt.tight_layout()
plt.show()
```



Bottom 5 Best Selling Pizzas - Total Sales

[46]:
```
pizzas_by_name = Pizza_data.groupby('pizza_name')['total_price'].sum()
bottom5 = pizzas_by_name.sort_values(ascending=True).head(5)

ax = bottom5.plot(kind='bar', figsize=(8,5), color='orange', edgecolor='black')
plt.title("Bottom 5 Pizzas Sold")
plt.xlabel("Pizza Name")
plt.ylabel("Total Revenue")
plt.xticks(rotation=45)

for i, val in enumerate(bottom5):
    plt.text(i, val + 2, str(val), ha='center', va='bottom', fontsize=9,␣
 ↪fontweight='bold')

plt.tight_layout()
plt.show()
```


Bottom 5 Pizzas Sold
Total Revenue
Pizza Name
11588.5
13955.75
15277.75
15360.5
15596.0
The Brie Carre Pizza
The Green Garden Pizza
The Spinach Supreme Pizza
The Mediterranean Pizza
The Spinach Pesto Pizza
0
2000
4000
6000
8000
10000
12000
14000
16000

[ ]:

[ ]:

[ ]:

[ ]: